# 09. object_oriented_programming

```
Data Sciences Institute
Python
```

# Learning objectives

- Be able to understand what object-oriented programming is

# What is object-oriented programming?

- Object-Oriented Programming (OOP) is a way of writing programs by organizing code around objects, which represent real-world things.
- Why do we need OOP? Imagine we want to store information about a car.
- Without OOP, we might write:

```
car_name = “Taylor”
car_brand = “Honda”
car_model = “Civic”
car_vin = "1VWAT7A32EC093779”
```

This works for one car — but what if we have 10 cars? 100? 1,000? Repeating variables like this quickly becomes messy and inefficient.

# Classes: Blueprints for objects

- OOP solves this by using **classes**.
- A **class** is a **blueprint** that describes what an object should look like.
- For example, we can create a Car class:

```
class Car:
    def __init__(self, name, brand, model, vin):
        self.name = name
        self.brand = brand
        self.model = model
        self.vin = vin
```

This creates a class called Car

__init__ runs automatically when we create a new car; it sets up the data that each car sill store

Self refers to the current car object

# Classes: Blueprints for objects

- Once we have a class, we can create **objects** from it

```
car1 = Car("Taylor", "Honda", "Civic",
"1VWAT7A32EC093779")

car2 = Car("Alex", "Toyota", "Corolla",
"2HGFB2F50FH512345")

#Accessing an attribute e.g. vin number
car1.vin
'1VWAT7A32EC093779'
```

Each object:
- Has the **same structure**
- Stores **different data**

# Class methods

- As we discussed, a method is a function that belongs to a class.
- Methods let objects do things, not just store data.

```
class Car:
    def __init__(self, name, brand, model, vin):
        self.name = name
        self.brand = brand
        self.model = model
        self.vin = vin
#define a method to describe the car
    def describe(self):
      print(f"{self.name} is a {self.brand} {self.model}")
car1.describe()
Taylor is a Honda Civic
```

# Quiz: OOP

1. **What is the correct way to create an object for this class?**

```
class Dog:
    def __init__(self, name):
        self.name = name
```

A. my_dog = Dog
B. my_dog = Dog(“Milo”)
C. my_dog = Dog.name(“Milo”)
D. my_dog = Dog.self(“Milo”)

# Quiz: OOP

**2. How do we print the dog's name?**

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
my_dog = Dog("Milo",3)
```

A. print(my_dog)
B. print(Dog.name)
C. print(my_dog.name)
D. print(my_dog.age)

# Quiz: OOP

**3. Fill in the blank to add a method that prints "Woof!" when called**

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

   def bark(self):
        ___________
```

# Exercise # 1

- Try this on your own:

  - Create a class called `Person`, and create a Person object with the following **Attributes**
    - first_name
    - middle_name
    - last_name
    - year_of_birth
  - **Methods**
    - full_name → Takes the first and last name and prints out the full name
    - initials → Takes the first character of the first and last name and prints out initials

# Making an attribute “private”

- So far, we’ve been able to access attributes directly:

```
#Accessing an attribute e.g. vin number
car1.vin
'1VWAT7A32EC093779'
```

- While this works, directly accessing attributes is often discouraged in Object-Oriented Programming.
- OOP encourages encapsulation — **keeping an object’s internal data protected and controlling how it is accessed or modified**.

# Getter and setter

- Getter → a method to access (“read”) the value of an attribute
- Setter → a method to change (“update”) the value of an attribute

We use getters are setters to control how data is accessed or changed, as well as check the values before setting them.

# Example without getters/setters

```
class Car:
    def __init__(self, name, brand, model, vin):
        self.name = name
        self.brand = brand
        self.model = model
        self.vin = vin
```

Here, these values can be set to anything, even invalid values like model = ASKDHAKSJHDAKS

# Encapsulation in Python

- In Python, Privacy is **by convention**, not enforced
  - We signal that an attribute should not be touched directly by adding _ or __.

```
class Car:
    def __init__(self, name, brand, model, vin):
        self.__vin = vin
```

  - Here, __vin is considered internal, we access it using methods instead of directly
  - We are saying *Please don't access _vin directly, use my methods instead.*"

# Add a getter (read-only access)

```
class Car:
    def __init__(self, name, brand, model, vin):
        self.name = name
        self.brand = brand
        self.model = model
        self._vin = vin

    def get_vin(self):
        return self._vin

car = Car("Taylor", "Honda", "Civic", "1VWAT7A32EC093779")
print(car.get_vin())
```

Getters and setters allow controlled access to an object's data, helping enforce rules and protect internal state.

# Add a setter (controlled updates)

```
class Car:
    def __init__(self, name, brand, model, vin):
        self.name = name
        self.brand = brand
        self.model = model
        self._vin = vin
    def get_vin(self):
        return self._vin
    def set_vin(self, new_vin):
        if len(new_vin) != 17:
                print("Invalid VIN length!")
        else:
                self._vin = new_vin
car.set_vin("ABC")
```

# Key points

- **Underscore (_ or __)** → signals “private/protected”
- **Getter/setter** → optional but recommended if you want **safe access or validation**
- You **don’t need a getter/setter** if you’re ok with letting everyone access it directly.

# Quiz: Private attributes, getters and setters

1. **What does a double underscore __ before an attribute name mean in Python?**

A. The attribute is deleted automatically

B. The attribute is private

C. The attribute cannot be used in the class

D. The attribute is public

# Quiz: Private attributes, getters and setters

**2. Why would you use a getter or setter for a private attribute?**

A. To make the code run faster

B. To safely access or modify the attribute

C. To automatically print the attribute

D. To convert the attribute to a list